

熱中症対策 見守りシステム

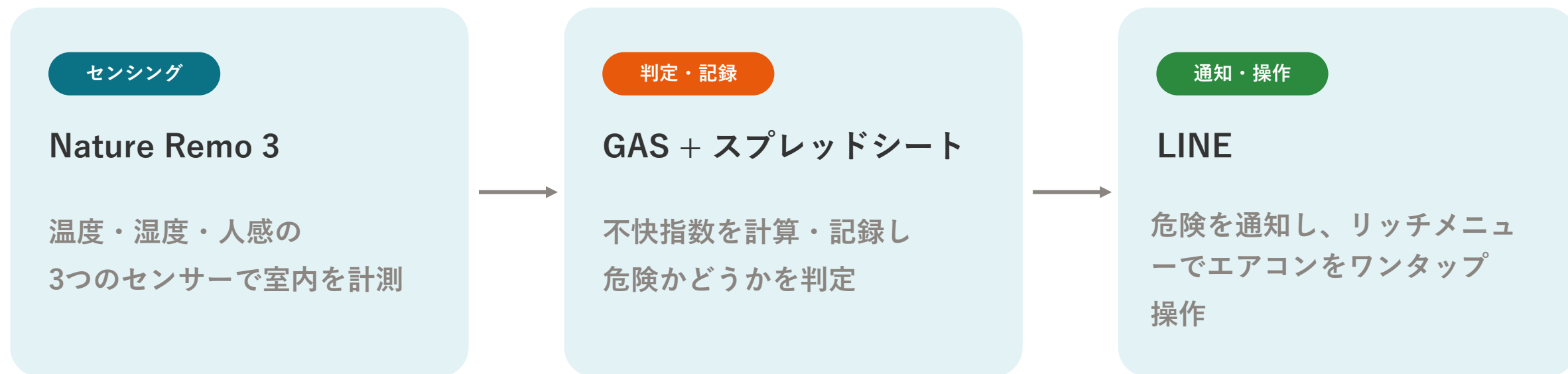
高齢者・子どもを見守る — Nature Remo 3 × LINE

組み込み実験 最終発表

グループ16 : 谷口義翔 ・ 平野汐音 ・ 二杉啓介 ・ 藤田航

システム概要

Nature Remo 3・スプレッドシート・LINE を連携し、室内環境の見守りから自動介入までを行う



危険を検知したら LINE で通知 → 30分応答がなければ自動で冷房を ON

デモ動画

02





エアコン

Remo

29.4° >80%



自動

送風

冷房

除湿

暖房



風量



上下風向

おやすみ
運転自動内部
クリーン

エアコン

Remo

29.4° >80%



自動

送風

冷房

除湿

暖房



風量



上下風向

おやすみ
運転自動内部
クリーンLINEから
あげる

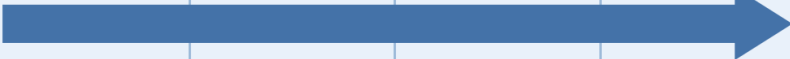
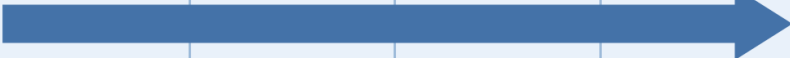
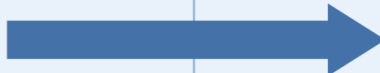
さげる



- 機能ごとにモジュールを分割し、複数個の作業対象にわけ、それぞれで作成することで分担して並列作業を行なった。
- それぞれで動作確認を行い、結合テストを行なった。
- 機能的に被っている部分があったのでリファクタリングを行なった。

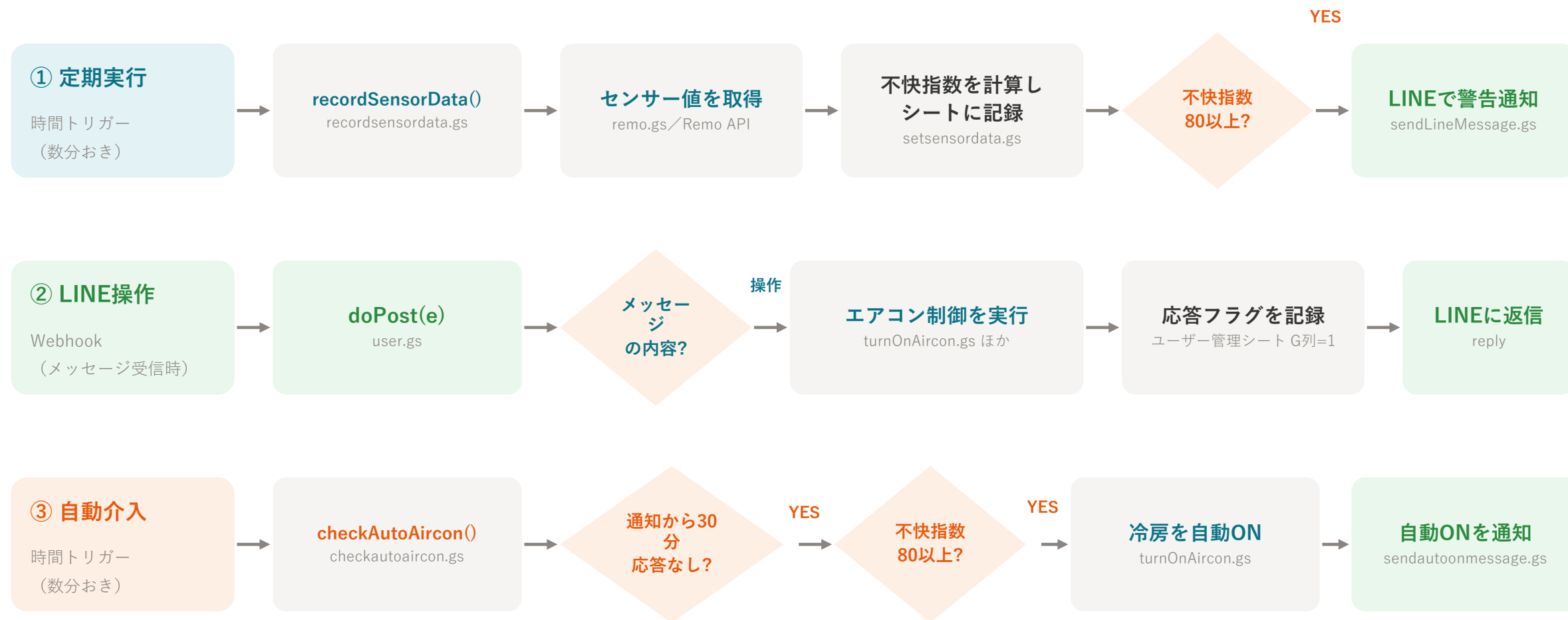
開発スケジュール

05

担当	タスク	6/17 4限	6/24 3限	6/24 4限	7/1 3限	7/1 4限	7/8 3限	7/8 4限	7/15 3限
全員	要求仕様・設計のみなおし								
谷口(sub) 藤田	LINE用プログラム (作成・テスト)								
谷口 平野(sub)	スプレッドシート 管理用プログラム (作成・テスト)								
二杉	Remo3データ取得 プログラム (作成・テスト)								
平野	エアコン操作 プログラム (作成・テスト)								
全員	結合テスト								
全員	発表資料作成								

処理全体の流れ（フローチャート）

06



スプレッドシート（sensor シート：計測値・不快指数 / ユーザー管理シート：userID・応答フラグ・通知時刻） — 3つの処理が読み書きして状態を共有

今回作成したgsファイルは以下の通り

A

センサーデータ収集・記録

recordsensordata.gs

remo.gs

setsensordata.gs

getsheet.gs

B

LINE通知

sendLineMessage.gs

sendautoonmessage.gs

C

LINE操作の受付

user.gs

lastMotion.gs

D

エアコン制御

turnOnAircon.gs

turnOnAirconDryMode.gs

turnOffAircon.gs

tempup.gs

tempdown.gs

getAirconid.gs

E

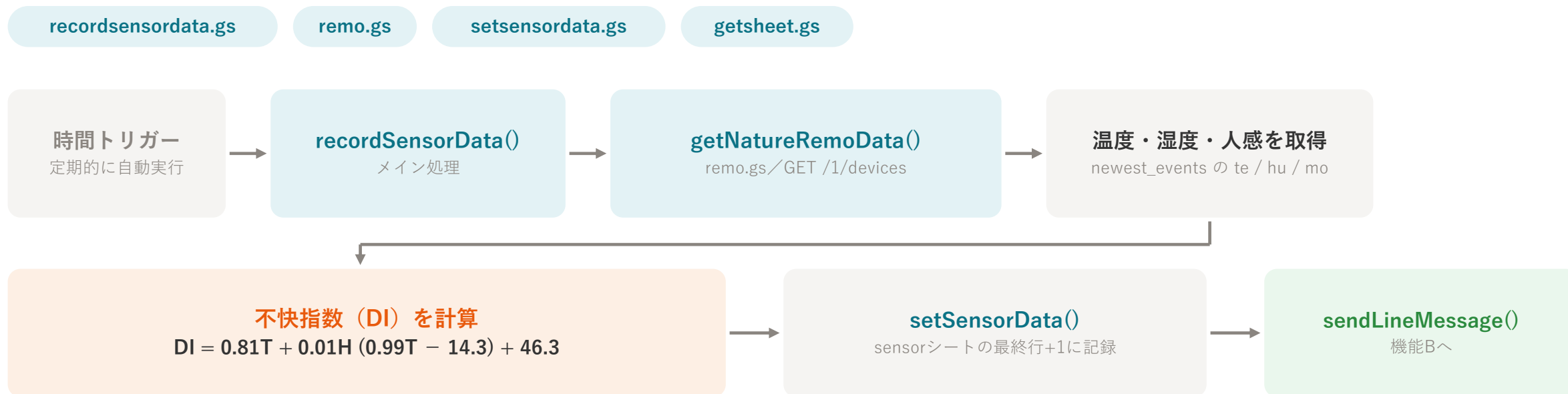
自動エアコンON判定

checkautoaircon.gs

機能A | センサーデータ収集・記録

08

時間トリガーで recordSensorData() が定期的に呼ばれる



- 記録内容は 5列：日時・温度(°C)・湿度(%)・人感・不快指数
- `remo.gs` の `getNatureRemoData()` は API 呼び出しの共通関数として全機能から利用
- `getsheet.gs` はシート取得と最終行取得の共通処理 (`getSheet` / `getLastData`)

機能B | LINE通知

09

不快指数が 80 以上のときだけ、登録ユーザー全員に警告を送る

sendLineMessage.gs

sendautoonmessage.gs



通知メッセージの内容

⚠ 不快指数が高くなっています。

温度・湿度・不快指数の現在値

水分補給のよびかけ

「30分以内に操作がなければ自動でエアコンON」の予告

sendAutoOnMessage()

機能E（自動介入）がエアコンを自動ONしたときに呼ばれる

「30分以内に反応がなかったため、自動でエアコンをONにしました。」

を全ユーザーに送信

The image shows a simulated LINE chat interface. A blue bubble from a bot contains a warning icon and the text: '不快指数が高くなっています。' followed by '温度：29.8℃', '湿度：72%', and '不快指数：81.4'. Below this, it says '熱中症に注意してください。水分補給をお願いします30分間、応答を待機しています。30分以内に「冷房」「除湿」「エアコンOFF」などの操作がない場合は、条件を満たしていれば自動でエアコンを起動します。' with a timestamp of 20:57. A grey bubble from a user responds with '30分以内に反応がなかったため、自動でエアコンをONにしました。' with a timestamp of 20:58.

機能C | LINE操作の受付

10

LINEのメッセージ受信（Webhook）で doPost(e) が呼ばれ、内容ごとに処理を振り分ける

user.gs

lastMotion.gs

メッセージ受信

LINE Webhook

doPost(e)

user.gs

メッセージ
の内容？

「とうろく」

操作コマンド

「今いる？」

その他

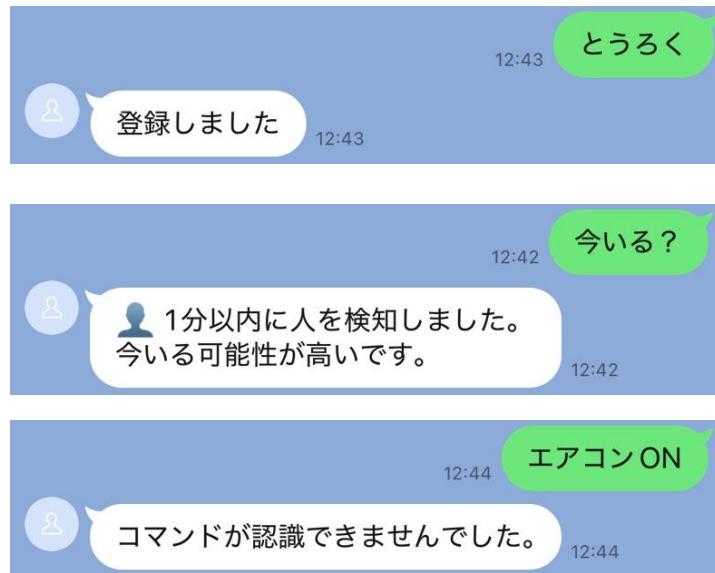
userID をユーザー管理シートに登録（重複チェックあり）→「登録しました」

エアコンOFF／さげる／あげる／冷房／除湿 → 機能Dの関数を呼び出し

getData() (lastMotion.gs) で人感センサーの最終検知からの経過秒数を確認 → 在室を推定して返信

「コマンドが認識できませんでした。」と返信

最後に replyToken を使って処理結果を LINE で返信（例：「冷房モードに設定しました。」）



機能D | エアコン制御

11

Nature Remo API の `aircon_settings` に POST してエアコンを操作する

`turnOnAircon.gs`

`turnOnAirconDryMode.gs`

`turnOffAircon.gs`

`tempup.gs`

`tempdown.gs`

`getAirconid.gs`

`turnOnAircon()`

冷房 25°C で ON

`operation_mode: cool,`
`temperature: 25`

`turnOnAirconDryMode()`

除湿モードで ON

`operation_mode: dry`

`turnOffAircon()`

電源 OFF

`button: power-off`

`tempUp()` / `tempDown()`

設定温度を $\pm 1^{\circ}\text{C}$

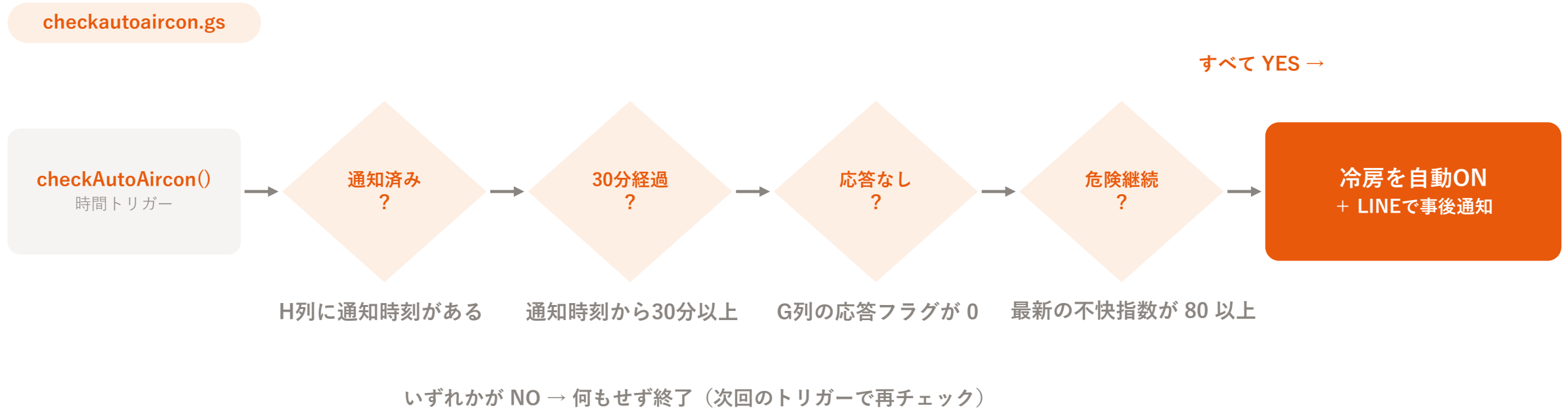
現在の設定温度を取得して ± 1

- `getAirconId()` : appliances 一覧から `type == "AC"` の家電を探して ID を返す（共通処理）
- 各操作の実行後は ユーザー管理シートの G列（応答フラグ）を 1 に更新 → 「応答あり」となり自動ONの対象外に
- API : POST `https://api.nature.global/1/appliances/{id}/aircon_settings`（アクセストークンで認証）

機能E | 自動エアコンON判定

12

時間トリガーで checkAutoAircon() が呼ばれ、4つの条件を順にチェックする



- 自動ON後は turnOnAircon() (機能D) と sendAutoOnMessage() (機能B) を呼び出す
- 判定終了後は G列・H列をリセットし、次の通知に備える
- 「通知 → 30分待つ → 自動介入」の二段階にすることで、勝手な操作を防ぎ本人の意思を尊重

ユーザインタフェース（LINEリッチメニュー）

13

高齢者・子どもでも迷わないよう、ふりがな付き・ワンタップ操作のリッチメニューを作成



ボタンを押すと対応するテキストが送信され、doPost(e) が処理を実行する（機能C）

スプレッドシートによる状態管理

2枚のシートをデータベースとして使い、機能間で状態を共有する

ユーザー管理シート

	A	B	C	D	E	F	G	H	I	J	K
1	日時	userID					返信なしかありか	通知時刻			
2	2026/06/27 10:4	U94440789b0679119f799631714d204e7						1 2026年7月8日水曜日 20時57分00秒			
3	2026/06/27 13:2	U6b7b141f31fc443797b4b78a13ada01f	こんにちは					1 2026年7月8日水曜日 20時57分00秒			
4	2026/07/01 14:3	Ufc30ddf9ea4cc1840a66ccced3bd4f73	エアコンON/OFF					1 2026年7月8日水曜日 20時57分01秒			
5	2026/07/01 14:3	U1dafdf547bd5821f1a59a0f9360eac0	エアコンON/OFF					1 2026年7月8日水曜日 20時57分02秒			
6											
7											
8											
9											
10											
11											
12											
13											
14											

登録日時・userID・応答フラグ（G列）・通知時刻（H列）を保持。

通知への「応答があったか」の判定に使う。

sensor シート

	A	B	C	D	E
1	時刻	温度	湿度	人感	不快指数
2	2026/07/08	30.4	75	1	82.8
3	2026/07/08	30.4	75	1	82.8
4	2026/07/08	30.4	75	1	82.8
5	2026/07/08	30.3	76	1	82.8
6	2026/07/08	30.9	74	1	83.4
7	2026/07/08	30.9	74	1	83.4
8	2026/07/08	31	72	1	83.2
9	2026/07/08	27.2	78	1	78.2
10	2026/07/08	26.8	80	1	77.8
11	2026/07/08	27.2	79	1	78.3
12	2026/07/08	27.9	79	1	79.4
13	2026/07/08	28.5	80	1	80.5
14	2026/07/08	30.6	80	1	83.9
15	2026/07/08	30.7	78	1	83.7
16	2026/07/08	30.5	76	1	83.1

日時・温度・湿度・人感・不快指数を1行ずつ蓄積。

最新行を参照して通知・自動介入の判定に使う。

温度、湿度、人感センサーの三つのデータを収集した

```
const discomfort =  
  0.81 * deviceData[0].newest_events.te.val +  
  0.01 * deviceData[0].newest_events.hu.val *  
  (0.99 * deviceData[0].newest_events.te.val - 14.3) +  
  46.3;
```

不快指数の計算

```
const arg = {  
  te: deviceData[0].newest_events.te.val,  
  hu: deviceData[0].newest_events.hu.val,  
  mo: deviceData[0].newest_events.mo.val,  
  di: Math.round(discomfort * 10) / 10  
};
```

実装できたところ

16

- LINEとエアコンとremo3の連携
- LINE通知
- エアコン操作をLINEのボタンで操作
- 通知後の処理

実装できなかったこと

17

- 一人だけしかまだ使えない
- Remo3のアクセストークンを取得するためにはユーザーから送るしかないため